Neural Network Track Extrapolation

for the LHCb Upgrade II

G. Scriven

LHCb · Nikhef

March 2026

Outline

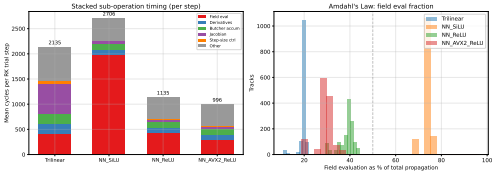
1. Motivation
2. Track Extrapolation & Runge–Kutta Schemes
3. Neural Network Magnetic Field Surrogate
4. Neural Network Full Extrapolation
5. Summary & Outlook

Why Now? — The LHCb Upgrade II Challenge

- LHCb Upgrade II: instantaneous luminosity increases to $\mathcal{L} = 1.5 \times 10^{34}$ cm^{-2}s^{-1}
- $\sim$5–10$\times$ more tracks per event
- Track extrapolation = propagating charged particles through the dipole magnet via Runge–Kutta integration
- **Current cost**: $\sim$8.35 μs/track
- Magnetic field lookups dominate: 76% of total time

$\Rightarrow$ Two strategies to accelerate:

1. **NN field surrogate**: replace field lookup with a cache-friendly neural network
2. **NN full extrapolation**: replace the entire RK integration with a single forward pass



RK sub-operation cost breakdown and Amdahl's Law projection for field acceleration

The Track State Vector

All LHCb extrapolators represent the track state at position z as:

$$\mathbf{s} = \begin{pmatrix} x \\ y \\ t_x \\ t_y \\ q/p \end{pmatrix} \quad \text{at position } z$$

- x, y: transverse positions (mm)
- $t_x = dx/dz$, $t_y = dy/dz$: direction tangents (slopes)
- q/p: charge / total momentum (MeV^{-1})
- Independent variable is z (beam axis), not time

Covariance propagation via the 5×5 transport matrix F:

$$C_{\text{out}} = F\, C_{\text{in}}\, F^T$$

Equations of Motion — Lorentz Force

A charged particle in field $\mathbf{B} = (B_x, B_y, B_z)$ obeys:

$$\frac{d}{dz}\begin{pmatrix} x \\ y \\ t_x \\ t_y \end{pmatrix} = \begin{pmatrix} t_x \\ t_y \\ (q/p)\, c_{\text{light}}\, \mathcal{A}_x \\ (q/p)\, c_{\text{light}}\, \mathcal{A}_y \end{pmatrix}$$

where $n = \sqrt{1 + t_x^2 + t_y^2}$ (arc-length correction) and:

$$\mathcal{A}_x = n\left[\, t_y(t_x B_x + B_z) - (1 + t_x^2)\, B_y \,\right]$$

$$\mathcal{A}_y = n\left[\, -t_x(t_y B_y + B_z) + (1 + t_y^2)\, B_x \,\right]$$

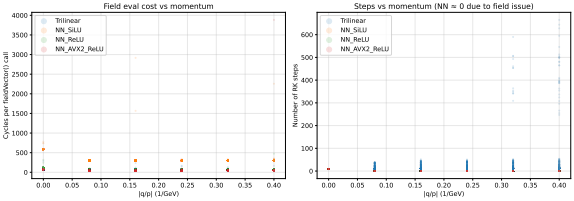
Physical interpretation

- Positions evolve linearly through slopes
- Slopes change due to the magnetic field — this is the bending
- Coupling $\propto (q/p) c_{\text{light}} \propto q/p$: low momentum particles bend more

The LHCb Dipole Field

The vertical field B_y is *not* uniform — it varies by a factor of $\sim 3\times$ across the magnet:

z (mm)	B_y (T)	Rel. curvature
0	1.10	$1.65\times$
2000	0.95	$1.43\times$
4000	0.70	$1.05\times$
6000	0.50	$0.75\times$
8000	0.40	$0.60\times$



Field evaluation cost varies with track momentum — low-p tracks require more RK steps.

Key constraint

Any model that cannot represent **position-dependent curvature** will inevitably fail.

Embedded Runge–Kutta Framework

An embedded RK method of order $p(p-1)$ computes **two estimates** per step, defined by its **Butcher tableau**:

$$
\begin{array}{c|cccc}
c_1 & & & & \\
c_2 & a_{21} & & & \\
\vdots & \vdots & \ddots & & \\
c_s & a_{s1} & \cdots & a_{s,s-1} & \\
\hline
 & b_1 & b_2 & \cdots & b_s \\
 & b_1^* & b_2^* & \cdots & b_s^*
\end{array}
$$

Stages: $\mathbf{k}_i = h\,\mathbf{f}\Big(z_n + c_i h,\ \mathbf{y}_n + \sum_{j<i} a_{ij}\,\mathbf{k}_j\Big)$

Update: $\mathbf{y}_{n+1} = \mathbf{y}_n + \sum_i b_i\,\mathbf{k}_i \quad (\text{order } p)$

Error: $\mathbf{e} = \sum_i (b_i - b_i^*)\,\mathbf{k}_i$

Cash–Karp 5(4) — The LHCb Default

6 stages, 5th-order with 4th-order error estimate:

$$
\begin{array}{c|cccccc}
0 & & & & & & \\
\frac{1}{5} & \frac{1}{5} & & & & & \\
\frac{3}{10} & \frac{3}{40} & \frac{9}{40} & & & & \\
\frac{3}{5} & \frac{3}{10} & -\frac{9}{10} & \frac{6}{5} & & & \\
1 & -\frac{11}{54} & \frac{5}{2} & -\frac{70}{27} & \frac{35}{27} & & \\
\frac{7}{8} & \frac{1631}{55296} & \frac{175}{512} & \frac{575}{13824} & \frac{44275}{110592} & \frac{253}{4096} & \\
\hline
b_i & \frac{37}{378} & 0 & \frac{250}{621} & \frac{125}{594} & 0 & \frac{512}{1771} \\
b_i^* & \frac{2825}{27648} & 0 & \frac{18575}{48384} & \frac{13525}{55296} & \frac{277}{14336} & \frac{1}{4}
\end{array}
$$

- Minimises leading truncation error of 5th-order formula
- $c_6 = 7/8$ (interior) gives more reliable step-size estimates
- Preferred over Fehlberg for non-stiff systems like tracking

The RK Scheme Zoo — 15 Methods Available

Scheme	Stages	Order	FSAL	Cost (field evals)
Cash–Karp (default)	6	5(4)	No	6/step
Dormand–Prince	7	5(4)	Yes	6/step (FSAL)
Bogacki–Shampine 3	4	3(2)	No	4/step
Bogacki–Shampine 5	8	5(4)	No	8/step
Heun–Euler	2	2(1)	No	2/step
Tsitouras 5	7	5(4)	Yes	6/step (FSAL)
Sharp–Verner 6	9	6(5)	Yes	8/step (FSAL)
Sharp–Verner 7	12	7(6)	Yes	11/step (FSAL)
Verner 6	8	6(5)	No	8/step
Verner 7	10	7(6)	No	10/step
Verner 8	13	8(7)	No	13/step
Verner 9	16	9(8)	No	16/step

- FSAL = First Same As Last: last stage of step n = first of $n+1$
- Higher-order methods take **larger steps** for the same tolerance $\Rightarrow$ fewer total field evaluations

Adaptive Step-Size Control

The step is accepted/rejected based on the normalised error:

$$E = \max\left(\frac{|e_x|}{\epsilon_x},\ \frac{|e_y|}{\epsilon_y},\ \frac{|e_{t_x}|}{\epsilon_{t_x} + |\Delta t_x|\,\epsilon_{\text{rel}}}\right)$$

- **Accept** if $E \leq \sigma$ (default $\sigma = 5.5$)
- Mixed absolute–relative criterion for slopes prevents overly small steps in high-curvature regions

PI controller for step scaling:

$$h_{n+1} = h_n \cdot \text{clamp}(f,\ 0.125,\ 4.0)\,, \quad f = \begin{cases} E^{-1/4} & \text{shrink (order } p-1) \\ E^{-1/5} & \text{grow (order } p) \end{cases}$$

Termination: curling ($|t_x|, |t_y| > 10$), excessive curvature ($|(q/p)B_y| > 1/\text{m}$), step count ≥ 1000, or out-of-bounds.

Jacobian Propagation — Variational Equations

The 5×5 transport matrix has the structure (q/p constant in RK step):

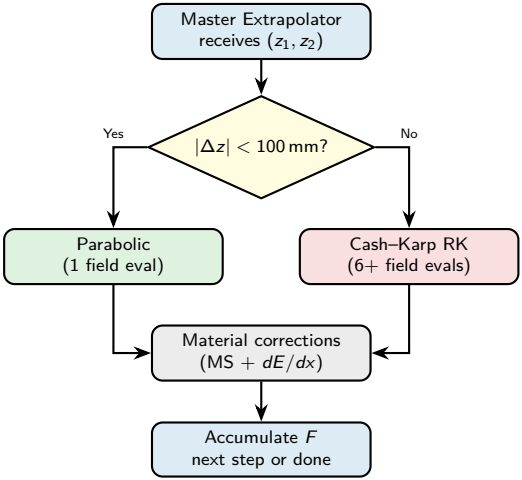
$$F = \begin{pmatrix} 1 & 0 & J_{x,t_x} & J_{x,t_y} & J_{x,q/p}c_{\text{light}} \\ 0 & 1 & J_{y,t_x} & J_{y,t_y} & J_{y,q/p}c_{\text{light}} \\ 0 & 0 & J_{t_x,t_x} & J_{t_x,t_y} & J_{t_x,q/p}c_{\text{light}} \\ 0 & 0 & J_{t_y,t_x} & J_{t_y,t_y} & J_{t_y,q/p}c_{\text{light}} \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

The slope Jacobian evolves as:

$$\frac{d}{dz}\begin{pmatrix} J_{t_x} \\ J_{t_y} \end{pmatrix} = \frac{q}{p}c_{\text{light}} \begin{pmatrix} \partial_{t_x}\mathcal{A}_x & \partial_{t_y}\mathcal{A}_x \\ \partial_{t_x}\mathcal{A}_y & \partial_{t_y}\mathcal{A}_y \end{pmatrix} \begin{pmatrix} J_{t_x} \\ J_{t_y} \end{pmatrix}$$

with position Jacobians obtained by integration: $J_x = \int J_{t_x}\, dz$. Solved simultaneously alongside the trajectory using the **same** RK stages.

Master Extrapolator — Production Dispatch



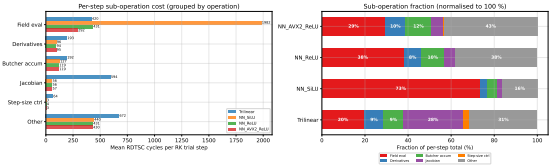
Stepping & material corrections:

1. Divide total Δz into steps ≤ 1000 mm
2. For each step:
   - Validate: $|x|, |y| < 10$ m, $|t_x|, |t_y| < 5$
   - Select sub-extrapolator
   - Propagate state
   - Accumulate: $F_{\text{tot}} = F_{\text{step}} \cdot F_{\text{prev}}$
3. If enabled, apply:
   - **Multiple scattering** (angular broadening)
   - **Energy loss** (dE/dx)
   - **Electron bremsstrahlung**

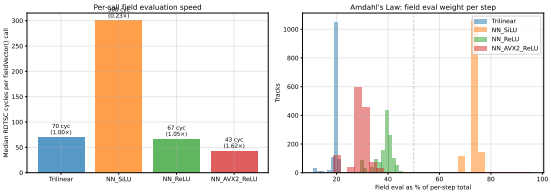
100 mm threshold

Parabolic: $\mathcal{O}(\Delta z^3)$ error, 1 field eval.
For $|\Delta z| < 100$ mm: error $\lesssim 10^{-4}$ mm.

Profiling: Where the Time Goes



RDTSC cycle-level cost breakdown per RK step.



Field call cost & Amdahl's law analysis.

Metric	**Value**
Field lookup cost	$\sim$0.109 μs per call (trilinear interpolation)
Lookups per track	$\sim$69 (CashKarp: $\sim$11.5 steps $\times$ 6 stages)
Field fraction	$\sim$76% of total propagation cost
Total per-track	$\sim$8.35 μs

Strategy: NN Field Map Surrogate

Idea

Replace trilinear interpolation on the $81 \times 81 \times 146$ grid ($\sim$4 MB) with a compact neural network that maps $(x, y, z) \to (B_x, B_y, B_z)$.

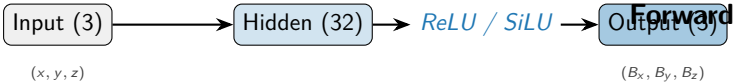
Why this works:

- The field map is **static per run** — ideal NN target
- The field $\mathbf{B}(x, y, z)$ is a smooth C^∞ function in vacuum $\Rightarrow$ well-approximated by a compact MLP (universal approximation theorem)
- **Amdahl's Law**: if field takes 76% of runtime and NN is $k\times$ faster, overall speedup $= 1/(0.24 + 0.76/k)$
  - $k = 2 \Rightarrow 1.6\times, \quad k = 5 \Rightarrow 2.5\times, \quad k = \infty \Rightarrow 4.2\times$

Key requirements:

- Accuracy: MAE < 0.1 Gauss (1 T $= 10{,}000$ Gauss)
- Speed: must beat trilinear ($\sim$57 FLOPs, but memory-bound)
- Size: must fit in L1 cache (≤ 32 KB)

Field NN Architecture: MLP $3 \to 32 \to 3$



Property	Value
Parameters	227
Memory	908 bytes (float32)
FLOPs (ReLU)	416
FLOPs (SiLU)	512

Forward pass with z-score normalisation:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu_{\text{in}}}{\sigma_{\text{in}}}$$

$$\mathbf{h} = \sigma\left(W_1\,\hat{\mathbf{x}} + \mathbf{b}_1\right)$$

$$\hat{\mathbf{y}} = W_2\,\mathbf{h} + \mathbf{b}_2$$

$$\mathbf{B} = \hat{\mathbf{y}} \cdot \sigma_{\text{out}} + \mu_{\text{out}}$$

Activations:

- ReLU: $\sigma(x) = \max(0, x)$
  — 1 FLOP/neuron, SIMD-friendly
- SiLU: $\sigma(x) = x/(1 + e^{-x})$
  — smoother, 4 FLOPs/neuron

Grid Search: 30 Architectures

Search space:

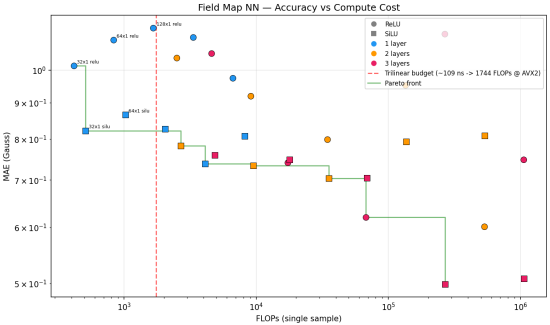
- Hidden widths: 32, 64, 128, 256, 512
- Depths: 1, 2, 3 layers
- Activations: ReLU, SiLU
- $= 5 \times 3 \times 2 = 30$ models

Training:

- Train on all 957,906 grid vertices
- Validate on 100K random off-grid points (ground truth = trilinear interpolation)
- Adam, cosine LR schedule, batch 4096

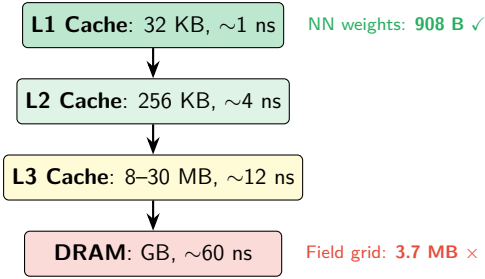
Accuracy targets:

- MAE < 0.1 Gauss
- p99 < 1.0 Gauss
- Max < 10.0 Gauss



Pareto frontier: accuracy vs computational cost.

L1/L2 Cache Analysis — The Key Speedup Mechanism



Trilinear interpolation:

- 8 corner lookups $\times$ 3 components
- Scattered access in ∼3.7 MB 3D grid
- Cache miss $\Rightarrow$ ∼60 ns latency
- Memory-bound, not compute-bound

NN field evaluation:

- 227 parameters $\times$ 4 bytes = **908 B**
- Fits **entirely** in L1 cache
- All accesses sequential (matrix–vector)
- Compute-bound (ALU-limited)

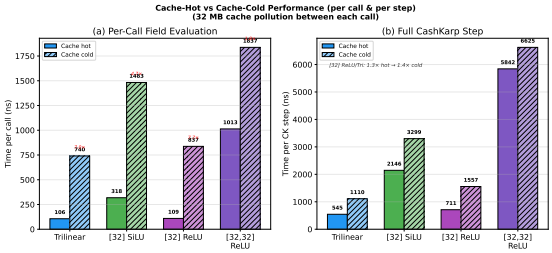
Key insight

The NN trades raw FLOP count (416 vs 57) for **cache locality**. With 69 field calls

Hot Cache vs Cold Cache — Real-World Impact

Component	Hot (ns)	Cold (ns)	Cold/ Hot
Per-call field evaluation:			
Trilinear	106	740	7.0×
NN [32] SiLU	318	1483	4.7×
NN [32] ReLU	109	837	7.7×
NN [32,32] ReLU	1014	1837	1.8×
Cash–Karp step (6 stages):			
Trilinear	545	1110	2.0×
NN [32] SiLU	2146	3299	1.5×
NN [32] ReLU	711	1557	2.2×
NN [32,32] ReLU	5842	6625	1.1×

C++ micro-benchmark, −O2.
Cold: 32 MB flush between calls.



Hot vs cold cache timing per CK step.

Key observations

- Trilinear: 7× cold penalty (DRAM latency)
- CK step amortises: first call warms cache for remaining 5
- NN vs trilinear ratio is *stable* hot or cold

AVX2 SIMD Implementation (1/2) — Inner Loop

Deployed in `FieldMapNNReLUWeights.h`. Process **8 neurons simultaneously** using 256-bit AVX2 registers:

Pseudo-code (from LHCb stack implementation)

```
// Weights stored transposed: w[input][neuron_block] for coalesced loads
__m256 vin0 = _mm256_set1_ps(x_norm);   // broadcast normalised x
__m256 vin1 = _mm256_set1_ps(y_norm);   // broadcast normalised y
__m256 vin2 = _mm256_set1_ps(z_norm);   // broadcast normalised z

for (int blk = 0; blk < 4; ++blk) {    // 4 blocks x 8 = 32 neurons
  __m256 vh = _mm256_load_ps(&bias[blk*8]);          // load 8 biases
  vh = _mm256_fmadd_ps(w_x[blk], vin0, vh);          // + w_x * x
  vh = _mm256_fmadd_ps(w_y[blk], vin1, vh);          // + w_y * y
  vh = _mm256_fmadd_ps(w_z[blk], vin2, vh);          // + w_z * z
  vh = _mm256_max_ps(vh, _mm256_setzero_ps());        // ReLU

  // Accumulate into 3 output neurons
  for (int j = 0; j < 3; ++j)
    acc[j] = _mm256_fmadd_ps(out_w[j][blk], vh, acc[j]);
}
// Horizontal sum: acc[j] -> scalar via _mm256_hadd_ps chain
```

AVX2 SIMD Implementation (2/2) — Batch Mode

`evaluate_relu_avx2_batch<N>`: processes N tracks with **shared weight register loads**:

Batch pseudo-code

```
template<int N>
void evaluate_relu_avx2_batch(const float inputs[N][3],
                              float outputs[N][3]) {
  for (int blk = 0; blk < 4; ++blk) {
    // Load weights ONCE per block (shared across N tracks)
    __m256 vw0 = _mm256_load_ps(&w_x[blk*8]);
    __m256 vw1 = _mm256_load_ps(&w_y[blk*8]);
    __m256 vw2 = _mm256_load_ps(&w_z[blk*8]);
    __m256 vb  = _mm256_load_ps(&bias[blk*8]);

    for (int t = 0; t < N; ++t) {          // Per-track evaluation
      __m256 vh = _mm256_fmadd_ps(vw0, broadcast(inputs[t][0]), vb);
      vh = _mm256_fmadd_ps(vw1, broadcast(inputs[t][1]), vh);
      vh = _mm256_fmadd_ps(vw2, broadcast(inputs[t][2]), vh);
      vh = _mm256_max_ps(vh, zero);         // ReLU
      accumulate_outputs(t, blk, vh);       // output FMAs
    }
  }
}
```

−O2 / −O3 Compiler Optimisation Effects

Feature	Effect on NN field eval	Effect on trilinear interp.
-O2 baseline	Standard inlining; loop opt	Standard inlining; branch pred.
-O3 auto-vectorise	Small function body (227 params) ⇒ **fully inlined & unrolled**	Branch-heavy logic ⇒ limited vectorisation benefit
Loop unrolling	4-iteration hidden-layer loop fully unrolled	Boundary checks prevent full unrolling
FMA fusion	$W\mathbf{x} + \mathbf{b}$ maps directly to `fmadd`	Scattered multiply-add patterns; less FMA opportunity
`-ffast-math`	Enables reciprocal $\sqrt{}$ approx for SiLU	Minimal benefit (no transcendentals)

Butcher accumulation in the RK stepper

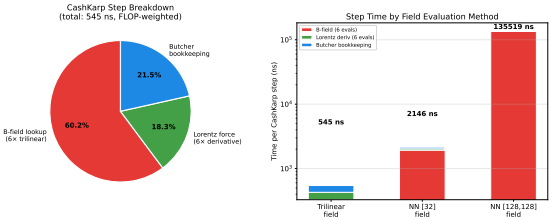
```
std::transform_reduce(..., std::execution::unseq, ...)
```

⇒ auto-vectorised by the compiler under -O3

Measured Cycle Costs: Component Breakdown

Component	−O2 (ns)	−O3 (ns)	vs. trilinear (−O3)
Trilinear interp.	106	116	1.0×
NN [32] SiLU	318	139	0.83×
NN [32] ReLU	109	40	**2.9**×
NN [32,32] ReLU	—	155	0.75×
Lorentz derivative	17	11	—

Per-call field evaluation.
C++ micro-benchmark, 100K iterations, AMD EPYC 7551P.



CK step breakdown: FLOPs (left) and measured timing per field method (right).

Component	FLOPs/step	Fraction
B-field (6×56)	336	60.2%
Lorentz deriv (6×17)	102	18.3%
Butcher bookkeeping	120	21.5%
Simplified step	**558**	100%

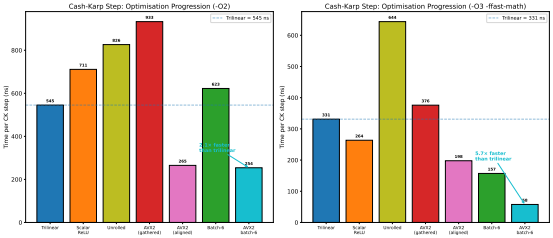
SiLU bottleneck

32 `exp()` calls = 80% of SiLU inference cost.
Switching to ReLU: 2.9× faster at −O3.

Optimisation Progression: CK Step Timing

Variant	−O2 (ns)	−O3 (ns)	vs. trilinear (−O3)
Trilinear	545	331	1.00×
NN scalar ReLU	711	264	0.80×
NN batch-6	623	157	0.47×
NN AVX2 (gathered)	933	376	1.14×
NN AVX2 (aligned)	265	198	0.60×
NN unrolled	826	644	1.94×
NN AVX2 batch-6	**254**	**58**	**0.17×**

CK step (6 stages, simplified kernel).
C++ micro-benchmark, 100K iterations.



CK step timing: optimisation progression.
Trilinear baseline shown as dashed line.

Failed optimisations

- AVX2 gathered (no pre-transposition): **slower**
- Manual unrolling: 2× **slower**

5.7× CK step speedup

AVX2 batch-6: 58 ns vs 331 ns (trilinear).
Exceeds Amdahl bound (2.5×) because batching restructures the *entire* step.

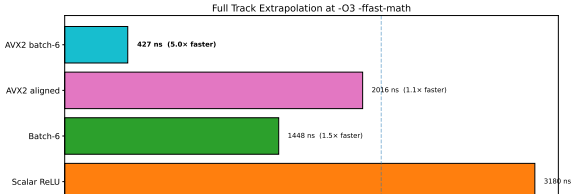
Full Track Timing & Production Comparison

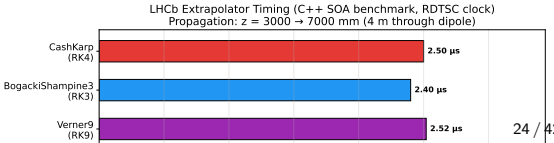
Variant	−O2 (ns)	−O3 (ns)	vs. trilinear (−O3)
Trilinear	2916	2141	1.00×
NN scalar ReLU	6658	3180	1.49×
NN batch-6	5813	1448	0.68×
NN AVX2 (aligned)	2520	2016	0.94×
NN AVX2 batch-6	**2375**	**427**	**0.20×**

10 simplified CK steps, z=3000→7000 mm.
C++ micro-benchmark, no Jacobian/adaptive stepping.

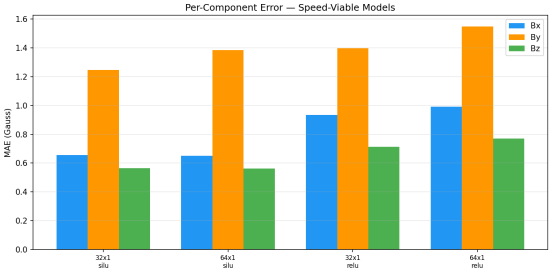
Extrapolator	Time (μs)	Pos. err (mm)
CashKarp RK5(4)	2.50	0.00
Bogacki–Shampine 3	2.40	0.10
Verner RK9(8)	2.52	0.08
Tsitouras RK5(4)	2.75	—
Herab RK5	1.95	5.10
Kisel polynomial	1.50	39.8

Full Gaudi/QMTest benchmark:
RDTSC clock, 121-track grid, z=3000→7000 mm.
Includes Jacobian, adaptive stepping, covariance.

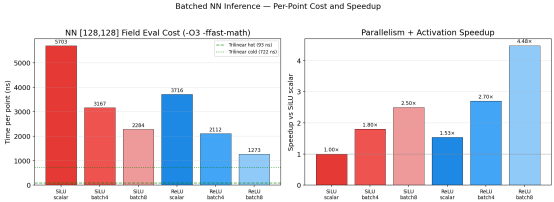




Field NN Results: Accuracy & Speed



Per-component field errors (B_x, B_y, B_z).



Batched inference throughput comparison.

Strategy: Replace the Entire RK Step

Concept

Train a neural network to learn the **full input–output mapping** of the Runge–Kutta integrator in a single forward pass.

Input (6 features):

$$\mathbf{x}_{\text{in}} = \left(x_0,\ y_0,\ t_{x0},\ t_{y0},\ q/p,\ \Delta z \right)$$

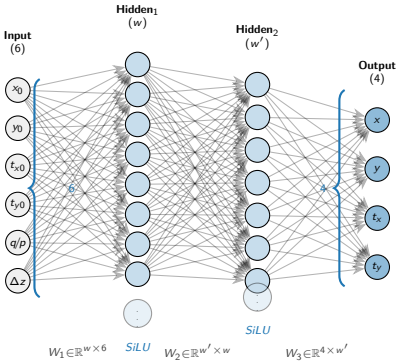
Output (4 features):

$$\mathbf{x}_{\text{out}} = \left(x,\ y,\ t_x,\ t_y \right) \quad \text{at} \quad z = z_0 + \Delta z$$

Training data: 50–100M trajectories generated by CashKarp RK4 integration through the real LHCb dipole field map.

Best approach found: a simple **multi-layer perceptron (MLP)** with SiLU activation and supervised MSE loss — no physics-informed constraints needed.

MLP Extrapolator Architecture



Forward pass (with z-score normalisation):

$$\hat{\mathbf{x}} = \frac{\mathbf{x}_{\text{in}} - \mu}{\sigma}$$

$$\mathbf{h}_1 = \sigma(W_1\,\hat{\mathbf{x}} + \mathbf{b}_1)$$

$$\mathbf{h}_2 = \sigma(W_2\,\mathbf{h}_1 + \mathbf{b}_2)$$

$$\hat{\mathbf{y}} = W_3\,\mathbf{h}_2 + \mathbf{b}_3$$

where $\sigma(x) = x/(1 + e^{-x})$ (SiLU).

Architecture presets:

Name	Layers	Params
single_256	[256]	~1.5K
single_512	[512]	~3K
shallow_256	[256, 256]	~101K
shallow_512_256	**[512, 256]**	**~402K**
medium	[256, 256, 128]	100K

Training & Loss Function

Loss function — pure supervised MSE:

$$\boxed{\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B}\|\hat{\mathbf{y}}_i(\theta) - \mathbf{y}_i^{\mathrm{RK4}}\|^2}$$

Expanded over output components:

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}\left[(\hat{x}_i - x_i)^2 + (\hat{y}_i - y_i)^2 + (\hat{t}_{x\,i} - t_{x\,i})^2 + (\hat{t}_{y\,i} - t_{y\,i})^2\right]$$

Why MSE?

- Targets $\mathbf{y}^{\mathrm{RK4}}$ are generated by CashKarp RK4 $\Rightarrow$ clean labels, no noise
- No physics-informed regularisation needed: the data *already encodes* the physics

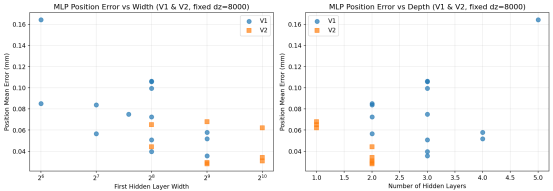
Training configuration:

Parameter	Value
Optimiser	Adam (β_1=0.9, β_2=0.999)
Learning rate	10^{-3} (initial)
LR schedule	Cosine annealing to 10^{-5}
Batch size	2048
Epochs	100 (early stopping)
Training samples	50–100M RK4 trajectories
Validation split	10%
Activation	SiLU
Weight init	Kaiming uniform

Data generation:

- Sample $(x_0, y_0, t_{x0}, t_{y0}, q/p, \Delta z)$ uniformly from the physical domain
- Propagate with CashKarp RK4 through the real LHCb dipole field map to obtain $\mathbf{y}^{\mathrm{RK4}}$
- Z-score normalise inputs *and* outputs

Architecture Selection: Shallow–Wide Wins



MLP width × depth comparison across 100+ trained models.

Key finding:

2-layer networks with 256–512 neurons **consistently outperform** 5-layer networks with 64 neurons — both in accuracy and inference speed.

Why?

- Shallow-wide: large matrix–vector products that saturate SIMD
- Deep-narrow: many small sequential operations, poor ILP
- The extrapolation mapping is smooth ⇒ a wide single layer captures most of the variation

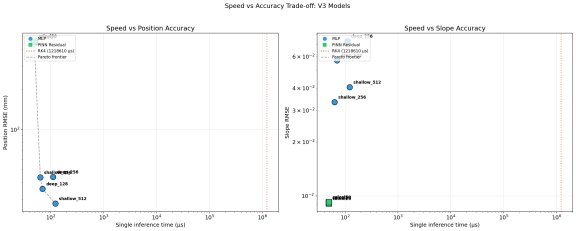
Best architecture

[512, 256] (2 hidden layers)

Results: Fixed $\Delta z = 8000$ mm

Model	Pos. (mm)	Time (μs)	Speedup
single_256	0.065	0.83	**3.0**×
shallow_256	0.044	1.50	1.7×
shallow_512_256	**0.028**	**1.93**	**1.3**×
shallow_1024_256	0.031	3.56	0.7×
C++ RK4	0	2.50	1.0×

V2 benchmark: MLP vs C++ CashKarp RK4.
Position error = mean RMSE across test set.



Speed vs accuracy Pareto frontier.

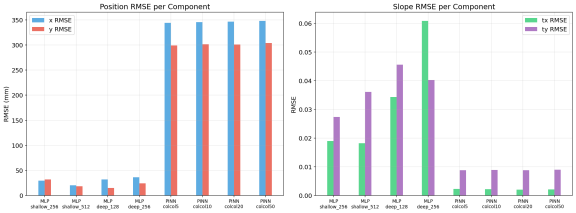
Key results

- **0.028 mm** position accuracy at **1.3**× speedup
- **0.065 mm** at **3.0**× speedup
- Sub-100μm accuracy across the board

Results: Variable $\Delta z \in [500, 12000]$ mm

Model	Params	Time (μs)	Pos. RMSE
RK4 (truth)	—	85.2	0
Linear	—	0.04	1.63 mm
Parabolic	—	0.09	1.63 mm
MLP deep_128	59K	65.0	1.01 mm
MLP shallow_256	101K	116.2	1.01 mm
MLP deep_256	233K	265.7	1.06 mm
MLP shallow_512	399K	465.6	0.96 mm

V3 C++ single-sample benchmark.
Variable Δz is $\sim$35$\times$ harder than fixed.

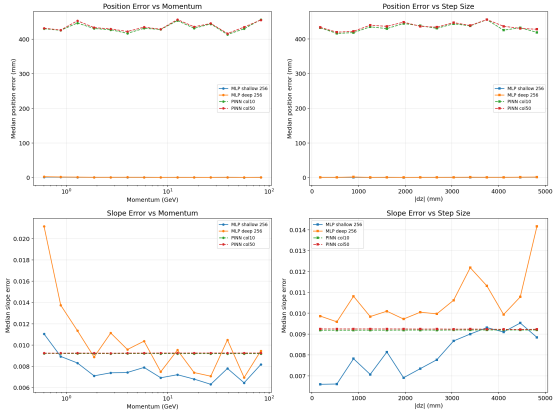


Per-component accuracy: x, y, t_x, t_y.

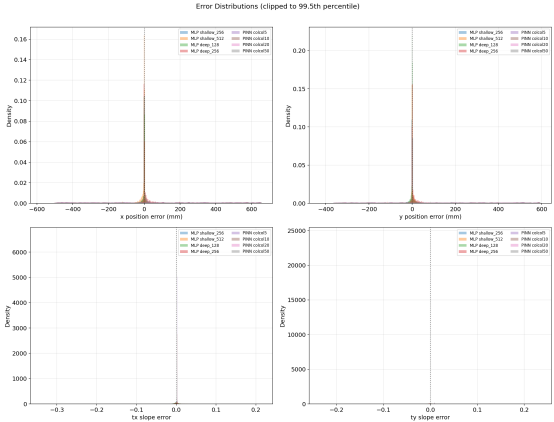
Variable Δz challenge

Position RMSE: 0.028 mm (fixed) $\rightarrow$ $\sim$1 mm (variable)
$=$ **35$\times$ harder** — the network must learn step-size–dependent curvature.

Error Analysis



Error vs kinematic variables (momentum, angles).



Residual distributions for the best MLP.

- Low-momentum tracks ($p < 5$ GeV) have larger errors due to stronger bending — the mapping is more nonlinear

C++ Deployment

Inference engine: `TrackMLPExtrapolator`

- **Eigen**-based matrix operations
- No TensorFlow / PyTorch dependency
- Supports ReLU, SiLU, Tanh, Sigmoid
- Input/output normalisation baked into model

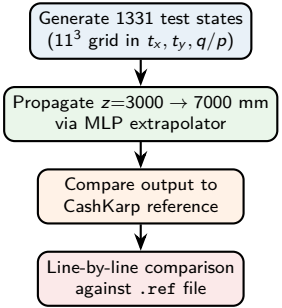
Binary model format (`.bin`):

1. Header: model type, number of layers
2. Normalisation: input/output mean & std
3. Per-layer: weight matrix + bias vector (float64)
4. Activation function name string

Gaudi integration:

- Configured via `ModelPath` property
- Drop-in replacement for `TrackRungeKuttaExtrapolator`
- Safety guards: max slope, max transverse

QMTest validation pipeline:



Platform support

Architecture-specific reference files for x86_64, ARM, optimised builds.

Summary: Two Complementary Strategies

Approach	Accuracy	Speed	Status	Upgrade II
Trilinear + RK4	exact (ref)	8.35 μs	Current production	Too slow
NN field + RK4	sub-Gauss	cache speedup	Deployed in stack	Ready
MLP (fixed Δz)	0.028 mm	1.93 μs	Validated	Promising
MLP (variable Δz)	$\sim$1 mm	$\sim$65 μs	Under optimisation	In progress

NN field map surrogate:

- 227-parameter MLP fits in L1 cache
- AVX2 SIMD: 8 neurons/cycle
- Sub-Gauss accuracy, negligible track impact
- Directly accelerates existing RK4 pipeline

MLP full extrapolation:

- Simple feedforward architecture
- 0.028 mm at 1.3$\times$ speedup (fixed Δz)
- Shallow-wide $>$ deep-narrow
- Variable Δz is 35$\times$ harder — active R&D

Outlook: Towards Upgrade II Readiness

1. **NN field map $\rightarrow$ production**
   - Integrate best architecture into Gaudi field service
   - Quantify real-event throughput improvement
   - Validate with full reconstruction chain
2. **MLP extrapolation: sub-mm at variable Δz**
   - V5 architecture sweep: wider networks, alternative activations
   - Momentum-binned specialists vs single universal model
   - Target: < 0.5 mm position RMSE at all step sizes
3. **Batched & SIMD inference for C++**
   - Apply AVX2 techniques from field NN to full extrapolator
   - Batch multiple tracks for matrix–matrix operations
   - Target: < 1 μs single-track inference
4. **Material corrections integration**
   - How to combine NN extrapolation with multiple scattering & dE/dx?
   - Hybrid approach: NN for field propagation + analytic material

Backup Slides

Backup: Normalisation Constants (Field NN)

	Mean	**Std**
x (mm)	0.0	2338.09
y (mm)	0.0	2338.09
z (mm)	6749.9995	4214.56
B_x (Gauss)	0.0	22.147
B_y (Gauss)	0.12293	71.609
B_z (Gauss)	0.0	22.147

Z-score normalisation constants for the deployed field NN (3→32→3 ReLU).

Backup: V2 Full MLP Results Table

Model	Architecture	Pos Mean (mm)	Pos 90th (mm)	Slope (mrad)	Time (μs)
shallow_1024_256	[1024, 256]	0.031	0.060	0.678	3.56
shallow_512_256	**[512, 256]**	**0.028**	**0.051**	**0.416**	**1.93**
shallow_512	[512, 512]	0.029	0.053	0.333	2.58
shallow_1024_512	[1024, 512]	0.034	0.063	0.783	4.71
shallow_256	[256, 256]	0.044	0.087	0.263	1.50
single_1024	[1024]	0.062	0.149	0.026	1.86
single_256	**[256]**	**0.065**	**0.135**	**0.017**	**0.83**
single_512	[512]	0.068	0.139	0.013	1.03
C++ RK4 (CashKarp)	—	0	0	0	2.50

Complete V2 MLP results (fixed $\Delta z = 8000$ mm). Best speed and best accuracy highlighted.

Backup: Other RK Schemes — Parabolic & STEP

Parabolic (single-step, $\mathcal{O}(\Delta z^3)$):

- 1 field evaluation at midpoint
- Treats trajectory as constant-curvature parabola
- Used for $|\Delta z| < 100$ mm (error $\lesssim 10^{-4}$ mm)

STEP (Runge–Kutta–Nyström):

- Exploits 2nd-order ODE structure: $d^2x/dz^2 = f(t_x, t_y, \mathbf{B})$
- 4 stages, only 2 field evaluations (stages 1 & 2 share position)
- Separate update formulae for positions and slopes:

$$x_{n+1} = x_n + h\, t_x + \frac{h^2}{6}(k_0 + k_1 + k_2)$$

$$t_{x,n+1} = t_x + \frac{h}{6}(k_0 + 2k_1 + 2k_2 + k_3)$$

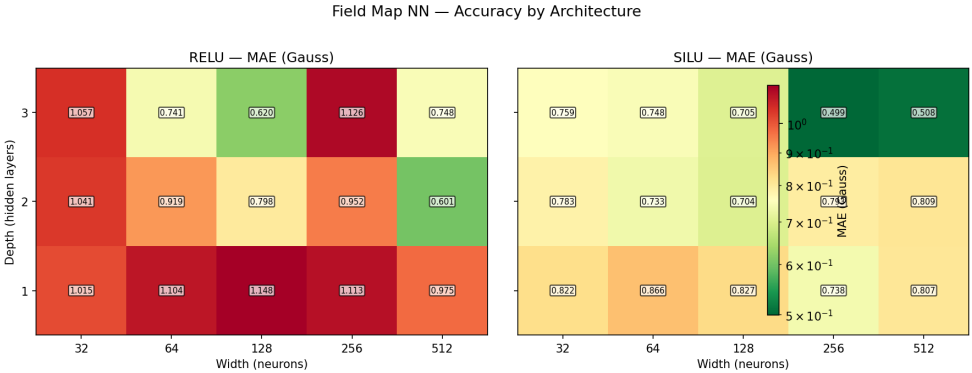
Backup: Dormand–Prince 5(4)

The most widely used adaptive RK scheme (MATLAB's `ode45`). Satisfies FSAL: only 6 function evaluations per accepted step.

$$c = \left(0,\ \tfrac{1}{5},\ \tfrac{3}{10},\ \tfrac{4}{5},\ \tfrac{8}{9},\ 1,\ 1\right)$$

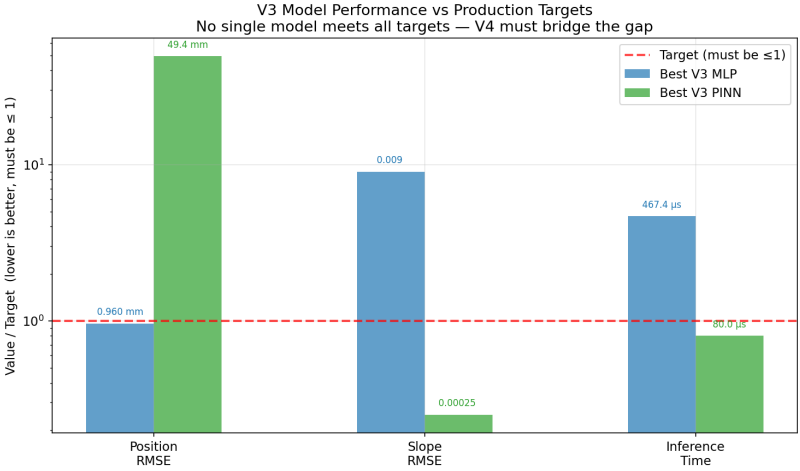
- 7 stages but last = first of next step (FSAL)
- Optimised to minimise local truncation error norm
- Comparable accuracy to Cash–Karp with 1 fewer evaluation per accepted step
- Available in the LHCb extrapolator via `RKScheme = "DormandPrince"`

Backup: Field NN Accuracy Heatmaps



Spatial distribution of field NN errors. Largest errors at detector edges ($|x|, |y| > 3500$ mm) where the field has steep gradients.

Backup: Production Readiness Assessment



Multi-metric production readiness assessment for MLP extrapolator models.